

Spring Boot 定时任务完整实现文档（终极版）

文档说明

本文档详细介绍了使用 Spring Boot 的 `@Scheduled` 注解实现两种典型定时任务的完整过程：

- 定点发送邮件** - 每年定时发送生日祝福邮件（需数据库存储用户信息）
- 定点更新天气数据** - 每5分钟获取实时天气数据并存储到数据库

目录

- 一、项目概述
- 二、技术架构
- 三、开发环境准备
- 四、项目依赖配置
- 五、配置文件详解
- 六、数据库设计与实现
- 七、实体类设计与实现
- 八、DTO类设计与实现
- 九、Mapper层实现
- 十、Service层实现
- 十一、定时任务实现
- 十二、控制器实现
- 十三、配置类实现
- 十四、主启动类实现
- 十五、Cron表达式详解
- 十六、fixedRate vs fixedDelay详解
- 十七、运行与测试
- 十八、常见问题与解决方案
- 十九、生产环境优化建议
- 二十、总结

一、项目概述

1.1 项目背景

在现代企业级应用中，定时任务是非常常见的需求。本文档基于 Spring Boot 框架，实现两个典型的定时任务场景：

- 场景一：生日祝福邮件** - 每年在用户生日当天自动发送祝福邮件
- 场景二：天气数据采集** - 每5分钟调用第三方天气API获取数据并存储到数据库

1.2 实现思路

生日邮件功能的实现思路：

text

1. 数据库设计阶段：
 - 设计**user**表存储用户基本信息（姓名、邮箱、生日日期）
 - 添加**last_sent_year**字段记录每年是否已发送，避免重复发送
 - 添加**enabled**字段控制是否启用该用户的生日提醒
2. 定时任务设计：
 - 使用Cron表达式"**0 0 9 * * ?**"指定每天上午**9:00**执行
 - 每次执行时查询当天过生日的用户
 - 比对**last_sent_year**与当前年份，未发送的才发送邮件
 - 发送成功后更新**last_sent_year**字段
3. 邮件服务设计：
 - 使用**Spring Boot Mail Starter**简化邮件发送
 - 邮件内容支持模板化，可动态替换用户姓名
 - 添加异常处理，单用户失败不影响其他用户

天气数据采集功能的实现思路：

text

1. 数据库设计阶段：
 - 设计**weather_data**表存储天气历史数据
 - 包含温度、湿度、风速、风向、天气描述等字段
 - 添加**query_time**记录数据查询时间，便于追溯
 - 添加索引优化查询性能
2. API调用设计：
 - 使用**Apache HttpClient**调用第三方天气**API**
 - 配置化**API**地址、**Key**、城市代码，便于切换
 - 添加超时设置和重试机制
 - 解析**JSON**响应数据
3. 数据存储逻辑：
 - 每次调用**API**后，将数据解析并插入数据库
 - 使用事务注解**@Transactional**保证数据一致性
 - 即使**API**调用失败，也记录错误信息到数据库
4. 定时任务设计：
 - 使用**fixedRate=300000**（5分钟）定时执行
 - 记录每次执行的耗时和结果
 - 添加数据清理任务，定期删除旧数据
5. 数据查询接口：
 - 提供**REST API**查询最新天气数据
 - 支持查询最近**24**小时天气趋势
 - 支持手动触发数据更新

1.3 功能列表

功能模块	具体功能	技术实现
生日邮件	每天9:00自动发送生日祝福	@Scheduled(cron) + JavaMailSender
天气采集	每5分钟获取并存储天气数据	@Scheduled(fixedRate) + HttpClient
数据清理	每天凌晨2点清理30天前数据	@Scheduled(cron) + MyBatis-Plus
数据查询	REST API查询天气数据	Spring MVC + MyBatis-Plus
手动更新	接口触发立即更新天气	REST API + Service调用

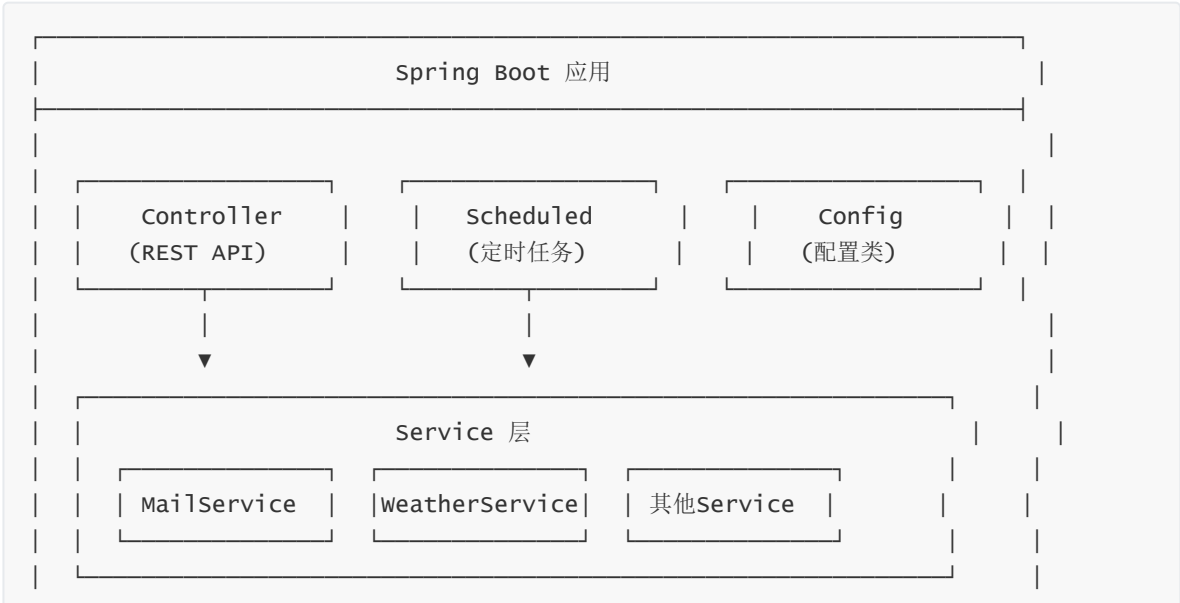
二、技术架构

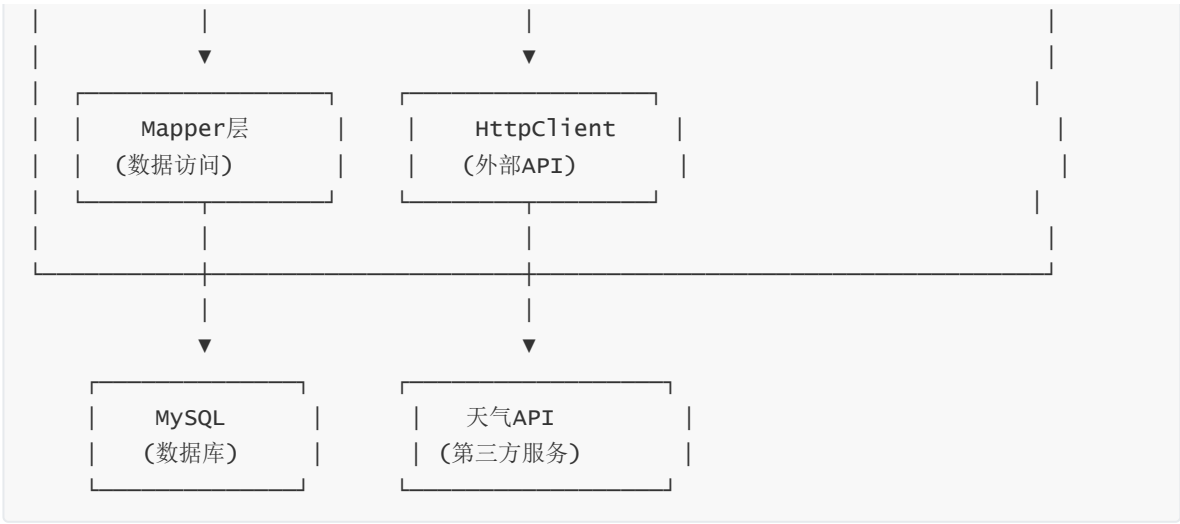
2.1 技术栈选型

技术	版本	用途
Spring Boot	2.7.0	基础框架
Java	11	开发语言
MyBatis-Plus	3.5.3	ORM框架
MySQL	8.0+	数据库
Apache HttpClient	4.5.14	HTTP客户端
Lombok	最新	简化代码
Jackson	2.13+	JSON解析
Maven	3.6+	构建工具

2.2 系统架构图

text

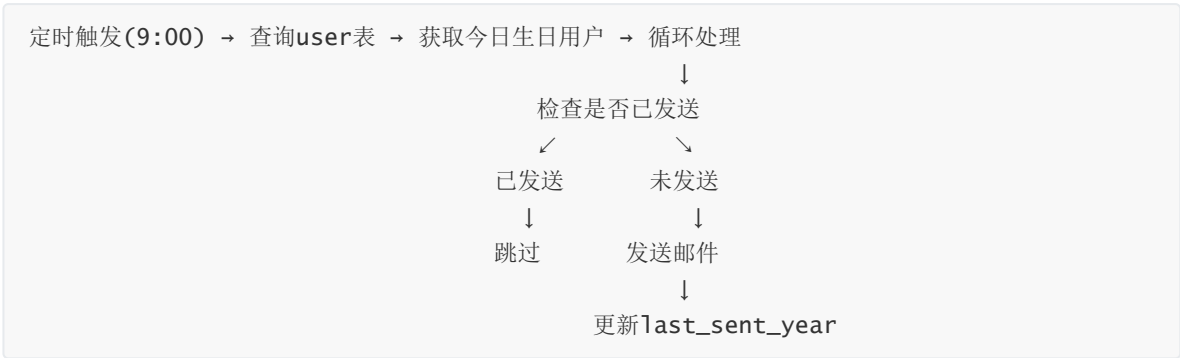




2.3 数据流图

生日邮件数据流：

text



天气数据数据流：

text



三、开发环境准备

3.1 环境要求

在开始开发之前，请确保您的开发环境满足以下要求：

环境	版本要求	验证命令
JDK	11+	<code>java -version</code>
Maven	3.6+	<code>mvn -version</code>
MySQL	8.0+	<code>mysql --version</code>
IDE	IntelliJ IDEA / Eclipse	-

3.2 创建Spring Boot项目

方式一：使用Spring Initializr（推荐）

1. 访问 <https://start.spring.io/>
2. 选择项目元数据：
 - Project: Maven
 - Language: Java
 - Spring Boot: 2.7.0
3. 填写项目信息：
 - Group: com.example
 - Artifact: scheduled-task-demo
4. 选择依赖：
 - Spring Web
 - Spring Boot DevTools
 - Lombok
 - MySQL Driver
 - MyBatis Framework
 - Java Mail Sender
5. 点击"Generate"下载项目压缩包
6. 解压后用IDE打开

方式二：使用IDE创建

以IntelliJ IDEA为例：

1. File → New → Project
2. 选择 Spring Initializr
3. 配置项目信息（同上）
4. 选择依赖（同上）
5. 点击 Finish 完成创建

3.3 初始化数据库

sql

```
-- 步骤1: 创建数据库
CREATE DATABASE IF NOT EXISTS scheduled_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

-- 步骤2: 使用数据库
USE scheduled_db;

-- 步骤3: 查看数据库状态
SELECT DATABASE(), NOW();
```

四、项目依赖配置

4.1 设计思路

Maven依赖配置是整个项目的基石，我们需要根据功能需求选择合适的依赖：

1. **Spring Boot Starter**：提供自动配置，简化集成
2. **MyBatis-Plus**：增强MyBatis，简化数据访问层开发
3. **Java Mail**：提供邮件发送能力
4. **HttpClient**：调用第三方HTTP接口
5. **Lombok**：减少样板代码（getter/setter等）

4.2 完整POM配置

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <!-- ===== 父依赖 ===== -->
    <!-- 继承Spring Boot父项目，获得版本管理和默认配置 -->
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.0</version>
        <relativePath/>
    </parent>

    <!-- ===== 项目基本信息 ===== -->
    <groupId>com.example</groupId>
    <artifactId>scheduled-task-demo</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>
    <name>scheduled-task-demo</name>
    <description>Spring Boot定时任务示例项目</description>
```

```

<!-- ===== 属性配置 ===== -->
<properties>
    <java.version>11</java.version>
    <mybatis-plus.version>3.5.3</mybatis-plus.version>
    <httpClient.version>4.5.14</httpClient.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<!-- ===== 依赖管理 ===== -->
<dependencies>

    <!--
        依赖1: Spring Boot Web Starter
        功能: 提供Web开发能力, 包括REST API、Tomcat嵌入等
        作用: 实现控制器, 对外提供天气数据查询接口
    -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!--
        依赖2: Spring Boot Mail Starter
        功能: 封装JavaMail, 简化邮件发送
        作用: 实现生日祝福邮件发送功能
    -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>

    <!--
        依赖3: Spring Boot Starter
        功能: Spring Boot核心启动器, 包含@Scheduled注解
        作用: 实现定时任务的核心依赖
    -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>

    <!--
        依赖4: MyBatis Plus Starter
        功能: 增强MyBatis, 提供Lambda查询、分页、代码生成等
        作用: 简化数据库操作, 减少SQL编写
    -->
    <dependency>
        <groupId>com.baomidou</groupId>
        <artifactId>mybatis-plus-boot-starter</artifactId>
        <version>${mybatis-plus.version}</version>
    </dependency>

    <!--
        依赖5: MySQL Connector
        功能: MySQL数据库驱动
        作用: 连接MySQL数据库
    -->
    <dependency>

```

```

        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>

    <!--
        依赖6: Lombok
        功能: 通过注解自动生成getter/setter/构造器等
        作用: 减少样板代码, 提高开发效率
        生命周期: provided表示仅在编译时需要
    -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <!--
        依赖7: Apache HttpClient
        功能: 提供HTTP客户端能力
        作用: 调用第三方天气API接口
    -->
    <dependency>
        <groupId>org.apache.httpcomponents</groupId>
        <artifactId>httpclient</artifactId>
        <version>${httpClient.version}</version>
    </dependency>

    <!--
        依赖8: Jackson Databind
        功能: JSON序列化/反序列化
        作用: 解析天气API返回的JSON数据
        注意: Spring Boot已包含此依赖, 此处显式声明版本确保兼容性
    -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
    </dependency>

    <!--
        依赖9: Spring Boot Test
        功能: 提供测试框架支持
        作用: 编写单元测试和集成测试
        生命周期: test表示仅在测试时需要
    -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

</dependencies>

<!-- ===== 构建配置 ===== -->
<build>
    <plugins>
        <!-- Spring Boot Maven插件: 打包可执行JAR -->
        <plugin>

```



```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
<configuration>
    <excludes>
        <!-- 打包时排除Lombok（编译时已用） -->
        <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
        </exclude>
    </excludes>
</configuration>
</plugin>

<!-- Maven编译插件：指定Java版本 -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <configuration>
        <source>${java.version}</source>
        <target>${java.version}</target>
        <encoding>${project.build.sourceEncoding}</encoding>
    </configuration>
</plugin>
</plugins>
</build>
</project>
```

4.3 依赖说明总结

依赖	必要原因	替代方案
spring-boot-starter-web	提供REST API能力	可不用，如仅需定时任务
spring-boot-starter-mail	发送邮件	JavaMail API直接使用
mybatis-plus-boot-starter	简化数据库操作	JPA、原生JDBC
httpclient	调用第三方API	RestTemplate、OkHttp
lombok	减少样板代码	手动编写getter/setter

五、配置文件详解

5.1 设计思路

配置文件遵循Spring Boot的约定优于配置原则：

1. **分层配置**：按功能模块拆分（数据库、邮件、天气、日志）
2. **外部化配置**：敏感信息不硬编码，通过配置文件管理
3. **环境隔离**：通过profile区分开发/生产环境

5.2 完整配置

```
# ===== 服务器配置 =====
server:
  port: 8080 # 应用端口号
  servlet:
    context-path: /api # 应用上下文路径（所有API前缀）

# ===== Spring 核心配置 =====
spring:
  # 数据库配置模块
  # 设计思路：使用MyBatis-Plus操作MySQL，配置连接池参数
  datasource:
    # JDBC连接URL
    # 参数说明：useSSL=false禁用SSL（本地开发），serverTimezone设置时区
    # characterEncoding设置字符集，allowPublicKeyRetrieval允许公钥检索
    url: jdbc:mysql://localhost:3306/scheduled_db?
useSSL=false&serverTimezone=Asia/Shanghai&characterEncoding=utf-
8&allowPublicKeyRetrieval=true
    username: root # 数据库用户名
    password: your_password # 数据库密码（请修改）
    driver-class-name: com.mysql.cj.jdbc.Driver # MySQL 8.0驱动
    # HikariCP连接池配置（Spring Boot默认连接池）
    hikari:
      maximum-pool-size: 10 # 最大连接数
      minimum-idle: 5 # 最小空闲连接数
      connection-timeout: 30000 # 连接超时时间（毫秒）
      idle-timeout: 600000 # 空闲超时时间（毫秒）
      max-lifetime: 1800000 # 连接最大生命周期（毫秒）

  # 邮件配置模块
  # 设计思路：配置邮件服务器参数，支持QQ、163等主流邮箱
  mail:
    # QQ邮箱示例（使用授权码）
    host: smtp.qq.com # SMTP服务器地址
    port: 587 # SMTP端口（587为TLS，465为SSL）
    username: your_email@qq.com # 发送邮箱账号
    password: your_authorization_code # QQ邮箱使用14位授权码（非登录密码）
    # 邮件协议属性
    properties:
      mail:
        smtp:
          auth: true # 启用认证
          starttls:
            enable: true # 启用TLS加密
          ssl:
            enable: false # 不使用SSL（与TLS二选一）
            connectiontimeout: 5000 # 连接超时
            timeout: 5000 # 读取超时
            writetimeout: 5000 # 写入超时

# ===== 业务配置模块 =====

# 天气API配置
# 设计思路：将API相关参数配置化，便于切换环境或修改城市
weather:
  api:
```

```
# 天气API地址（24小时预报）
url: https://pc2yvythc5.re.qweatherapi.com/v7/weather/24h
# API密钥（请使用实际有效的密钥）
key: 773eefa30eb64fde9137d8a77f404724
# 城市代码（101190101代表南京）
location: 101190101
# 城市名称（便于日志和显示）
location-name: 南京市
# 连接超时时间（毫秒）
connect-timeout: 10000
# 读取超时时间（毫秒）
read-timeout: 10000
```

生日邮件配置

设计思路：邮件内容模板化，支持动态替换

birthday:

mail:

邮件主题

subject: "生日快乐! 🎂"

邮件正文模板（%s会被用户名替换）

template: |

亲爱的 %s,

今天是你的生日，祝你生日快乐！愿你健康快乐每一天！

愿你在新的一年里：

- 工作顺利，事业有成
- 身体健康，心情愉快
- 家庭幸福，万事如意

此致

祝福团队

===== 定时任务配置 =====

scheduled:

定时任务线程池配置

task:

pool:

size: 10 # 线程池大小

prefix: scheduled-task- # 线程名称前缀

数据清理配置

clean:

weather:

retention-days: 30 # 天气数据保留天数

cron: 0 0 2 * * ? # 清理任务执行时间（每天凌晨2点）

===== MyBatis-Plus配置 =====

mybatis-plus:

实体类扫描包路径

type-aliases-package: com.example.entity

Mapper XML文件位置

mapper-locations: classpath*:mapper/**/*.xml

全局配置

global-config:

db-config:

主键类型：AUTO表示数据库自增

id-type: auto

逻辑删除字段名

```

    logic-delete-field: deleted
    # 逻辑删除值（1表示已删除）
    logic-delete-value: 1
    # 逻辑未删除值（0表示未删除）
    logic-not-delete-value: 0
# 配置项
configuration:
    # 开启驼峰命名自动映射
    map-underscore-to-camel-case: true
    # 开启SQL日志输出（开发环境）
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl

# ===== 日志配置 =====
logging:
    # 日志级别配置
    level:
        # 项目包日志级别: DEBUG
        com.example: DEBUG
        # MyBatis SQL日志级别: DEBUG
        com.example.mapper: DEBUG
        # Apache HttpClient日志级别: WARN（减少噪音）
        org.apache.http: WARN
    # 日志输出格式
    pattern:
        # 控制台输出格式: 时间 + 线程 + 级别 + 类名 + 消息
        console: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
        # 文件输出格式（如果配置了文件）
        file: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
    # 日志文件配置（可选）
    file:
        name: logs/application.log      # 日志文件路径
        max-size: 10MB                 # 单个日志文件最大大小
        max-history: 30                 # 保留天数

```

5.3 多环境配置（进阶）

生产环境建议使用多环境配置：

yaml

```

# application-dev.yml（开发环境）
spring:
    datasource:
        url: jdbc:mysql://localhost:3306/scheduled_db
        username: dev_user
        password: dev_password

# application-prod.yml（生产环境）
spring:
    datasource:
        url: jdbc:mysql://prod-db-host:3306/scheduled_db
        username: prod_user
        password: ${DB_PASSWORD} # 使用环境变量，不硬编码

```

```
# 启动时指定环境
# java -jar app.jar --spring.profiles.active=prod
```

六、数据库设计与实现

6.1 设计思路

用户表 (user) 设计:

1. **需求分析**: 需要存储用户的基本信息和生日日期
2. **字段设计**:
 - `id`: 主键, 自增, 唯一标识
 - `name`: 用户姓名, 用于邮件称呼
 - `email`: 邮箱地址, 用于接收邮件
 - `birthday`: 生日日期, 使用DATE类型
 - `last_sent_year`: 记录最后发送年份, 防止重复发送
 - `enabled`: 是否启用, 可临时禁用某个用户
3. **索引设计**: 在birthday字段上建立索引, 优化日期查询

天气数据表 (weather_data) 设计:

1. **需求分析**: 存储每次获取的天气数据, 支持历史查询
2. **字段设计**:
 - 基础信息: `id`、`location` (城市代码)、`location_name` (城市名称)
 - 天气数据: `temperature` (温度)、`humidity` (湿度)、`wind_speed` (风速)
 - 时间信息: `fx_time` (预报时间)、`query_time` (查询时间)、`created_at` (创建时间)
 - 原始数据: `raw_data` (JSON格式), 便于调试
3. **索引设计**: 组合索引(location, query_time)优化查询

6.2 完整建表SQL

sql

```
-- ===== 步骤1: 创建数据库 =====
-- 设计思路: 使用utf8mb4字符集支持emoji表情 (生日邮件中的蛋糕emoji)
CREATE DATABASE IF NOT EXISTS scheduled_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

-- 切换数据库
USE scheduled_db;

-- ===== 步骤2: 创建用户表 =====
-- 设计思路: 存储用户基本信息, 支持生日邮件发送
-- 表名: user (单数形式, 符合MyBatis-Plus默认映射规则)
DROP TABLE IF EXISTS `user`;
CREATE TABLE `user` (
    -- 主键字段: BIGINT类型, 自增长
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键ID, 自增长',
```

```

-- 用户基本信息
`name` VARCHAR(50) NOT NULL COMMENT '用户姓名',
`email` VARCHAR(100) NOT NULL COMMENT '邮箱地址（用于接收邮件）',

-- 生日相关字段
`birthday` DATE NOT NULL COMMENT '生日日期（格式：YYYY-MM-DD）',
`birthday_year` INT DEFAULT NULL COMMENT '出生年份（可选，用于计算年龄）',

-- 业务控制字段
`last_sent_year` VARCHAR(4) DEFAULT NULL COMMENT '最后一次发送邮件的年份（如：2024）',
`enabled` TINYINT DEFAULT 1 COMMENT '是否启用：1-启用，0-禁用',

-- 审计字段（自动填充）
`created_at` DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
`updated_at` DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',

-- 索引设计：提升查询性能
INDEX idx_birthday (`birthday`),           -- 生日查询索引
INDEX idx_enabled (`enabled`),             -- 启用状态索引
INDEX idx_email (`email`)                  -- 邮箱索引
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户生日表';

-- ===== 步骤3：创建天气数据表 =====
-- 设计思路：存储每次定时任务获取的天气数据
DROP TABLE IF EXISTS `weather_data`;
CREATE TABLE `weather_data` (
-- 主键字段
`id` BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键ID',

-- 地理位置信息
`location` VARCHAR(50) NOT NULL COMMENT '城市代码（如：101190101）',
`location_name` VARCHAR(50) DEFAULT NULL COMMENT '城市名称（如：南京）',

-- 天气数据字段
`temperature` DECIMAL(5,2) DEFAULT NULL COMMENT '温度（摄氏度）',
`humidity` INT DEFAULT NULL COMMENT '相对湿度（百分比）',
`wind_speed` DECIMAL(6,2) DEFAULT NULL COMMENT '风速（km/h）',
`wind_direction` VARCHAR(20) DEFAULT NULL COMMENT '风向（如：东南风）',
`weather_desc` VARCHAR(100) DEFAULT NULL COMMENT '天气描述（如：多云、小雨）',

-- 时间信息
`fx_time` DATETIME DEFAULT NULL COMMENT '预报时间（API返回的预报时间点）',
`query_time` DATETIME NOT NULL COMMENT '数据查询时间（本地时间）',

-- 原始数据（用于调试和问题排查）
`raw_data` TEXT COMMENT 'API返回的原始JSON数据',

-- 审计字段
`created_at` DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '记录创建时间',

-- 索引设计：优化查询性能
INDEX idx_location (`location`),           -- 城市索引
INDEX idx_query_time (`query_time`),       -- 查询时间索引
INDEX idx_fx_time (`fx_time`),             -- 预报时间索引
INDEX idx_location_query (`location`, `query_time`) -- 复合索引（城市+时间）

```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='天气
数据历史表';

-- ===== 步骤4: 插入测试数据 =====
-- 设计思路: 插入测试用户数据, 5月7日生日用于测试
INSERT INTO `user` (`name`, `email`, `birthday`, `enabled`) VALUES
('张三', 'zhangsan@example.com', '1990-05-07', 1),
('李四', 'lisi@example.com', '1988-05-07', 1),
('王五', 'wangwu@example.com', '1995-06-15', 1),
('赵六', 'zhao Liu@example.com', '1992-12-25', 1),
('测试用户', 'test@example.com', '2000-01-01', 0); -- 禁用的用户

-- 验证数据插入
SELECT * FROM `user`;

-- ===== 步骤5: 创建存储过程 (可选) =====
-- 设计思路: 创建存储过程用于定期清理旧数据
DELIMITER $$
CREATE PROCEDURE `clean_old_weather_data`(IN `days_to_keep` INT)
BEGIN
    -- 删除指定天数之前的天气数据
    DELETE FROM `weather_data`
    WHERE `query_time` < DATE_SUB(NOW(), INTERVAL days_to_keep DAY);

    -- 返回删除的行数
    SELECT ROW_COUNT() AS `deleted_count`;
END$$
DELIMITER ;

-- 调用示例:
-- CALL clean_old_weather_data(30);

```

6.3 索引优化说明

表名	索引名	索引列	优化场景
user	idx_birthday	birthday	查询今日过生日用户
user	idx_enabled	enabled	过滤启用的用户
weather_data	idx_location	location	按城市查询天气
weather_data	idx_query_time	query_time	按时间范围查询
weather_data	idx_location_query	location, query_time	城市+时间范围查询

6.4 数据示例

sql

```

-- 查询今日过生日用户
SELECT * FROM `user`
WHERE `enabled` = 1
AND DATE_FORMAT(`birthday`, '%m-%d') = DATE_FORMAT(NOW(), '%m-%d');

```

```
-- 查询最新天气数据
SELECT * FROM `weather_data`
WHERE `location` = '101190101'
ORDER BY `query_time` DESC
LIMIT 1;

-- 查询天气趋势（最近24小时平均温度）
SELECT
    DATE_FORMAT(`query_time`, '%Y-%m-%d %H:00:00') as hour,
    AVG(`temperature`) as avg_temp,
    AVG(`humidity`) as avg_humidity
FROM `weather_data`
WHERE `query_time` >= DATE_SUB(NOW(), INTERVAL 24 HOUR)
GROUP BY DATE_FORMAT(`query_time`, '%Y-%m-%d %H:00:00')
ORDER BY `hour` DESC;
```

七、实体类设计与实现

7.1 设计思路

实体类是数据库表在Java中的映射，遵循以下设计原则：

1. **使用Lombok**：通过注解自动生成getter/setter，减少样板代码
2. **与数据库字段对齐**：字段名采用驼峰命名，自动映射下划线字段
3. **使用包装类型**：使用BigDecimal/Integer等包装类型，避免默认值问题
4. **审计字段自动填充**：使用MyBatis-Plus的自动填充功能

7.2 User实体类

java

```
/**
 * 用户实体类
 *
 * 实现思路：
 * 1. 使用@Data注解：自动生成getter、setter、toString、equals、hashCode
 * 2. 使用@TableName注解：指定对应的数据库表名
 * 3. 使用@TableId注解：标记主键字段，type指定主键生成策略
 * 4. 使用@TableField注解：标记非表字段或特殊映射
 *
 * 对应数据库表：user
 */
package com.example.entity;

import com.baomidou.mybatisplus.annotation.*;
import lombok.Data;
import java.time.LocalDate;
import java.time.LocalDateTime;

@Data // Lombok注解：自动生成getter、setter、toString、equals、hashCode
@TableName("user") // 指定数据库表名（可省略，默认类名小写）
public class User {
```



```
/**
 * 主键ID
 *
 * 实现思路:
 * - IdType.AUTO: 使用数据库自增主键
 * - 对应数据库字段: id
 */
@TableId(type = IdType.AUTO)
private Long id;

/**
 * 用户姓名
 *
 * 实现思路:
 * - 使用@TableField注解可指定特殊映射（此处默认驼峰转下划线）
 * - NOT NULL约束: 数据库层面保证非空
 */
private String name;

/**
 * 邮箱地址
 *
 * 实现思路:
 * - 用于接收生日祝福邮件
 * - 建议在业务层做邮箱格式校验
 */
private String email;

/**
 * 生日日期
 *
 * 实现思路:
 * - 使用LocalDate类型（只包含年月日，不包含时间）
 * - 格式: YYYY-MM-DD
 */
private LocalDate birthday;

/**
 * 出生年份
 *
 * 实现思路:
 * - 可选字段，用于计算用户年龄
 * - 使用Integer包装类型，允许为null
 */
private Integer birthdayYear;

/**
 * 最后一次发送邮件的年份
 *
 * 实现思路:
 * - 用于避免同一年重复发送生日邮件
 * - 格式: 四位数字字符串，如"2024"
 */
private String lastSentYear;

/**
 * 是否启用
 *
```

```

    * 实现思路:
    * - 1: 启用, 接收邮件通知
    * - 0: 禁用, 不接收邮件通知
    * - 默认值为1
    */
    private Integer enabled;

    /**
     * 创建时间
     *
     * 实现思路:
     * - @TableField(fill = FieldFill.INSERT): 插入时自动填充
     * - 配合自动填充处理器 (MetaObjectHandler) 使用
     */
    @TableField(fill = FieldFill.INSERT)
    private LocalDateTime createdAt;

    /**
     * 更新时间
     *
     * 实现思路:
     * - @TableField(fill = FieldFill.INSERT_UPDATE): 插入和更新时自动填充
     * - 数据库ON UPDATE CURRENT_TIMESTAMP也会更新
     */
    @TableField(fill = FieldFill.INSERT_UPDATE)
    private LocalDateTime updatedAt;
}

```

7.3 WeatherData实体类

java

```

/**
 * 天气数据实体类
 *
 * 实现思路:
 * 1. 映射数据库weather_data表
 * 2. 使用BigDecimal处理浮点数 (避免精度问题)
 * 3. 保留原始JSON数据用于调试
 *
 * 对应数据库表: weather_data
 */
package com.example.entity;

import com.baomidou.mybatisplus.annotation.*;
import lombok.Data;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Data
@TableName("weather_data")
public class WeatherData {

    /**
     * 主键ID

```

```

*
* 实现思路：数据库自增主键
*/
@TableId(type = IdType.AUTO)
private Long id;

/**
* 城市代码
*
* 实现思路：
* - 如：101190101代表南京
* - 支持多城市扩展
*/
private String location;

/**
* 城市名称
*
* 实现思路：
* - 便于日志输出和界面展示
* - 避免每次都查城市代码映射表
*/
private String locationName;

/**
* 温度（摄氏度）
*
* 实现思路：
* - 使用BigDecimal：精确表示浮点数，避免double精度问题
* - precision=5, scale=2：最多5位整数，2位小数
*/
private BigDecimal temperature;

/**
* 湿度（百分比）
*
* 实现思路：
* - 范围：0-100
* - 使用Integer：湿度不需要小数精度
*/
private Integer humidity;

/**
* 风速（km/h）
*
* 实现思路：
* - 使用BigDecimal：风速可能有小数
*/
private BigDecimal windSpeed;

/**
* 风向
*
* 实现思路：
* - 如：东风、南风、西风、北风等
* - 存储字符串而非编码，便于阅读
*/
private String windDirection;

```

```

/**
 * 天气描述
 *
 * 实现思路:
 * - 如: 晴、多云、小雨、大雨
 * - 存储描述文本, 便于前端展示
 */
private String weatherDesc;

/**
 * 预报时间
 *
 * 实现思路:
 * - API返回的预报时间点
 * - 可能比查询时间晚(预报的是未来天气)
 */
private LocalDateTime fxTime;

/**
 * 查询时间
 *
 * 实现思路:
 * - 实际调用API的本地时间
 * - NOT NULL: 必须记录
 */
private LocalDateTime queryTime;

/**
 * 原始数据
 *
 * 实现思路:
 * - 存储API返回的完整JSON
 * - 用于问题排查和字段扩展
 */
private String rawData;

/**
 * 创建时间
 *
 * 实现思路: 记录插入数据库的时间
 */
@TableField(fill = FieldFill.INSERT)
private LocalDateTime createdAt;
}

```

7.4 MetaObjectHandler (自动填充处理器)

java

```

/**
 * MyBatis-Plus自动填充处理器
 *
 * 实现思路:
 * 1. 实现MetaObjectHandler接口

```

```

* 2. 重写insertFill和updateFill方法
* 3. 在实体类的@TableField注解中指定fill策略
*
* 作用：自动填充created_at、updated_at等审计字段
*/
package com.example.config;

import com.baomidou.mybatisplus.core.handlers.MetaObjectHandler;
import lombok.extern.slf4j.Slf4j;
import org.apache.ibatis.reflection.MetaObject;
import org.springframework.stereotype.Component;
import java.time.LocalDateTime;

@Slf4j
@Component
public class MyMetaObjectHandler implements MetaObjectHandler {

    /**
     * 插入时的填充策略
     *
     * 实现思路：
     * - 当实体字段标记@TableField(fill = FieldFill.INSERT)时
     * - 此方法会被自动调用
     */
    @Override
    public void insertFill(MetaObject metaObject) {
        log.debug("自动填充插入字段...");

        // 填充创建时间
        this.strictInsertFill(metaObject, "createdAt", LocalDateTime.class,
            LocalDateTime.now());

        // 填充更新时间（插入时也填充）
        this.strictInsertFill(metaObject, "updatedAt", LocalDateTime.class,
            LocalDateTime.now());
    }

    /**
     * 更新时的填充策略
     *
     * 实现思路：
     * - 当实体字段标记@TableField(fill = FieldFill.UPDATE)时
     * - 此方法会被自动调用
     */
    @Override
    public void updateFill(MetaObject metaObject) {
        log.debug("自动填充更新字段...");

        // 填充更新时间
        this.strictUpdateFill(metaObject, "updatedAt", LocalDateTime.class,
            LocalDateTime.now());
    }
}

```

八、DTO类设计与实现

8.1 设计思路

DTO (Data Transfer Object) 用于在不同层之间传输数据:

1. **API响应封装**: 统一响应格式
2. **外部接口映射**: 映射第三方API响应结构
3. **业务数据封装**: 封装业务处理结果

8.2 WeatherInfo DTO

java

```
/**
 * 天气信息DTO
 *
 * 实现思路:
 * 1. 封装天气数据, 供业务层使用
 * 2. 使用@Builder注解: 支持建造者模式构建对象
 * 3. 提供toString方法便于日志输出
 *
 * 用途: Service层返回天气数据给Controller或Task
 */
package com.example.dto;

import lombok.Builder;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@Builder // 提供建造者模式: weatherInfo.builder().temperature("25").build()
public class WeatherInfo {

    /**
     * 是否获取成功
     *
     * 实现思路:
     * - 用于区分正常数据和错误数据
     * - 前端根据此字段决定展示逻辑
     */
    @Builder.Default
    private boolean success = true;

    /**
     * 错误信息
     *
     * 实现思路:
     * - success为false时, 此字段有值
     * - 记录具体错误原因
     */
    private String errorMsg;

    /**
     * 温度 (摄氏度)
     *
     */
}
```

```

    * 实现思路：字符串类型，保留原始格式
    */
    private String temperature;

    /**
     * 湿度（百分比）
     */
    private String humidity;

    /**
     * 风速（km/h）
     */
    private String windSpeed;

    /**
     * 天气描述
     */
    private String weatherDesc;

    /**
     * 预报时间
     *
     * 实现思路：API返回的时间字符串
     */
    private String fxTime;

    /**
     * 查询时间
     *
     * 实现思路：本地时间，用于记录查询时刻
     */
    private LocalDateTime queryTime;

    /**
     * 重写toString方法
     *
     * 实现思路：
     * - 提供格式化的字符串，便于日志输出
     * - 区分成功和失败两种格式
     */
    @Override
    public String toString() {
        if (!success) {
            return String.format("天气获取失败：%s", errorMsg);
        }
        return String.format("当前天气 - 温度：%s°C，湿度：%s%%，风速：%skm/h，天气：%s，预报时间：%s",
            temperature, humidity, windSpeed, weatherDesc, fxTime);
    }
}

```

8.3 WeatherResponse DTO（API响应映射）

java

```

/**
 * 天气API响应结构DTO
 *
 * 实现思路:
 * 1. 与第三方API的JSON结构完全对应
 * 2. 使用@JsonProperty注解处理JSON字段名差异
 * 3. 只定义需要的字段, 忽略无关字段
 *
 * API返回JSON示例:
 * {
 *   "code": "200",
 *   "updateTime": "2024-05-07T10:00+08:00",
 *   "hourly": [
 *     {
 *       "fxTime": "2024-05-07T10:00+08:00",
 *       "temp": "23",
 *       "humidity": "65",
 *       "windSpeed": "12",
 *       "windDir": "东南风",
 *       "text": "多云"
 *     }
 *   ]
 * }
 */
package com.example.dto;

import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Data;
import java.util.List;

@Data
public class WeatherResponse {

    /**
     * 响应状态码
     *
     * 实现思路:
     * - "200"表示成功
     * - 其他值表示失败
     */
    @JsonProperty("code") // JSON中的字段名是"code"
    private String code;

    /**
     * 数据更新时间
     *
     * 实现思路: API数据的官方更新时间
     */
    @JsonProperty("updateTime")
    private String updateTime;

    /**
     * 天气预报链接
     *
     * 实现思路: 可用于生成详情页链接
     */
    @JsonProperty("fxLink")
    private String fxLink;

```



```

/**
 * 逐小时天气预报列表
 *
 * 实现思路:
 * - 通常返回24小时数据
 * - 我们取第一条作为"当前天气"
 */
@JsonProperty("hourly")
private List<Hourlyweather> hourly;

/**
 * 逐小时天气预报内部类
 *
 * 实现思路:
 * - 使用static内部类, 与外部类解耦
 * - 只定义需要的字段
 */
@Data
public static class Hourlyweather {

    /**
     * 预报时间
     */
    @JsonProperty("fxTime")
    private String fxTime;

    /**
     * 温度
     */
    @JsonProperty("temp")
    private String temp;

    /**
     * 湿度
     */
    @JsonProperty("humidity")
    private String humidity;

    /**
     * 风速
     */
    @JsonProperty("windSpeed")
    private String windSpeed;

    /**
     * 风向
     */
    @JsonProperty("windDir")
    private String windDir;

    /**
     * 天气状况描述
     */
    @JsonProperty("text")
    private String text;
}
}

```

8.4 统一响应结果封装

java

```
/**
 * 统一API响应结果封装
 *
 * 实现思路:
 * 1. 统一前后端数据交互格式
 * 2. 包含状态码、消息、数据三个部分
 * 3. 提供静态工厂方法简化创建
 *
 * 使用示例:
 * - Result.success(data) 返回成功响应
 * - Result.error(message) 返回失败响应
 */
package com.example.dto;

import lombok.Data;

@Data
public class Result<T> {

    /** 状态码: 200表示成功, 其他表示失败 */
    private int code;

    /** 响应消息 */
    private String message;

    /** 响应数据 */
    private T data;

    /** 时间戳 */
    private long timestamp;

    private Result() {
        this.timestamp = System.currentTimeMillis();
    }

    /**
     * 成功响应 (无数据)
     */
    public static <T> Result<T> success() {
        Result<T> result = new Result<>();
        result.setCode(200);
        result.setMessage("success");
        return result;
    }

    /**
     * 成功响应 (带数据)
     */
    public static <T> Result<T> success(T data) {
        Result<T> result = new Result<>();

```

```

        result.setCode(200);
        result.setMessage("success");
        result.setData(data);
        return result;
    }

    /**
     * 失败响应
     */
    public static <T> Result<T> error(String message) {
        Result<T> result = new Result<>();
        result.setCode(500);
        result.setMessage(message);
        return result;
    }

    /**
     * 自定义状态码的失败响应
     */
    public static <T> Result<T> error(int code, String message) {
        Result<T> result = new Result<>();
        result.setCode(code);
        result.setMessage(message);
        return result;
    }
}

```

九、Mapper层实现

9.1 设计思路

Mapper层负责数据库访问，遵循以下原则：

1. **继承BaseMapper**：获得基础的CRUD方法
2. **自定义查询**：使用@Select注解编写自定义SQL
3. **参数传递**：使用#{param}方式，防止SQL注入
4. **索引利用**：SQL语句充分利用数据库索引

9.2 UserMapper实现

java

```

/**
 * 用户Mapper接口
 *
 * 实现思路：
 * 1. 继承MyBatis-Plus的BaseMapper，获得基础CRUD能力
 * 2. 添加@Mapper注解，让Spring扫描并创建代理对象
 * 3. 定义自定义查询方法
 *
 * 基础方法（继承自BaseMapper）：
 * - insert()：插入用户
 * - deleteById()：按ID删除

```

```

* - updateById(): 按ID更新
* - selectById(): 按ID查询
* - selectList(): 条件查询列表
*/
package com.example.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.example.entity.User;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import org.apache.ibatis.annotations.Select;
import org.apache.ibatis.annotations.Update;
import java.time.LocalDate;
import java.util.List;

@Mapper // 标记为MyBatis Mapper接口
public interface UserMapper extends BaseMapper<User> {

    /**
     * 查询今天过生日的用户
     *
     * 实现思路:
     * 1. 使用DATE_FORMAT函数提取月-日进行比较
     * 2. 只查询enabled=1的用户
     * 3. 利用birthday索引提升查询性能
     *
     * SQL语句分析:
     * - DATE_FORMAT(birthday, '%m-%d'): 将生日转为"MM-DD"格式
     * - DATE_FORMAT("#{today}", '%m-%d'): 将今天转为"MM-DD"格式
     * - 比较两者是否相等
     *
     * @param today 今天的日期
     * @return 今天过生日的用户列表
     */
    @Select("SELECT * FROM user WHERE enabled = 1 " +
            "AND DATE_FORMAT(birthday, '%m-%d') = DATE_FORMAT("#{today}", '%m-%d')")
    List<User> findTodayBirthdayUsers(@Param("today") LocalDate today);

    /**
     * 更新用户最后一次发送邮件的年份
     *
     * 实现思路:
     * 1. 发送成功后更新last_sent_year字段
     * 2. 避免同一年重复发送
     *
     * @param userId 用户ID
     * @param year 发送年份 (如"2024")
     */
    @Update("UPDATE user SET last_sent_year = #{year} WHERE id = #{userId}")
    void updateLastSentYear(@Param("userId") Long userId, @Param("year") String year);

    /**
     * 根据邮箱查询用户 (高级功能示例)
     *
     * 实现思路: 用于登录或验证场景
     */
}

```

```

    @Select("SELECT * FROM user WHERE email = #{email} AND enabled = 1")
    User findByEmail(@Param("email") String email);

    /**
     * 统计本月过生日的用户数量
     *
     * 实现思路：用于统计分析
     */
    @Select("SELECT COUNT(*) FROM user WHERE enabled = 1 " +
            "AND MONTH(birthday) = MONTH(#{date})")
    int countBirthdayThisMonth(@Param("date") LocalDate date);
}

```

9.3 WeatherMapper实现

java

```

/**
 * 天气数据Mapper接口
 *
 * 实现思路：
 * 1. 继承BaseMapper获取基础CRUD方法
 * 2. 定义时间范围查询、最新数据查询等方法
 * 3. 使用Lambda表达式构建复杂查询条件
 */
package com.example.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.example.entity.WeatherData;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import org.apache.ibatis.annotations.Select;
import java.time.LocalDateTime;
import java.util.List;

@Mapper
public interface WeatherMapper extends BaseMapper<WeatherData> {

    /**
     * 查询指定城市最新的天气数据
     *
     * 实现思路：
     * 1. 按查询时间降序排序
     * 2. 取第一条记录
     * 3. 利用idx_location_query复合索引
     *
     * @param location 城市代码
     * @return 最新的天气数据
     */
    @Select("SELECT * FROM weather_data " +
            "WHERE location = #{location} " +
            "ORDER BY query_time DESC " +
            "LIMIT 1")
    WeatherData findLatestByLocation(@Param("location") String location);
}

```

```

/**
 * 查询指定时间范围内的天气数据
 *
 * 实现思路:
 * 1. 按时间范围筛选
 * 2. 按查询时间降序排序
 * 3. 用于趋势分析
 *
 * @param location 城市代码
 * @param startTime 开始时间
 * @param endTime 结束时间
 * @return 时间范围内的天气数据列表
 */
@Select("SELECT * FROM weather_data " +
        "WHERE location = #{location} " +
        "AND query_time BETWEEN #{startTime} AND #{endTime} " +
        "ORDER BY query_time DESC")
List<WeatherData> findByTimeRange(@Param("location") String location,
        @Param("startTime") LocalDateTime
startTime,
        @Param("endTime") LocalDateTime endTime);

/**
 * 查询最近N条天气记录
 *
 * 实现思路:
 * 1. 限制返回记录数
 * 2. 用于展示最近几天数据
 *
 * @param location 城市代码
 * @param limit 限制条数
 * @return 最近N条记录
 */
@Select("SELECT * FROM weather_data " +
        "WHERE location = #{location} " +
        "ORDER BY query_time DESC " +
        "LIMIT #{limit}")
List<WeatherData> findRecentRecords(@Param("location") String location,
        @Param("limit") int limit);

/**
 * 统计指定时间范围内的平均温度
 *
 * 实现思路:
 * 1. 使用聚合函数AVG
 * 2. 返回平均温度
 *
 * @param location 城市代码
 * @param startTime 开始时间
 * @param endTime 结束时间
 * @return 平均温度
 */
@Select("SELECT AVG(temperature) FROM weather_data " +
        "WHERE location = #{location} " +
        "AND query_time BETWEEN #{startTime} AND #{endTime} " +
        "AND temperature IS NOT NULL")
Double getAverageTemperature(@Param("location") String location,
        @Param("startTime") LocalDateTime startTime,

```

```
        @Param("endTime") LocalDateTime endTime);  
    }  
}
```

9.4 Mapper XML配置（复杂SQL示例）

当SQL语句较复杂时，建议使用XML方式：

xml

```
<!-- src/main/resources/mapper/WeatherMapper.xml -->  
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"  
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">  
<mapper namespace="com.example.mapper.WeatherMapper">  
  
    <!-- 天气趋势查询（按小时分组） -->  
    <select id="getHourlyTrend" resultType="java.util.Map">  
        SELECT  
            DATE_FORMAT(query_time, '%Y-%m-%d %H:00:00') as hour,  
            ROUND(AVG(temperature), 1) as avg_temp,  
            ROUND(AVG(humidity), 0) as avg_humidity,  
            MAX(wind_speed) as max_wind_speed  
        FROM weather_data  
        WHERE location = #{location}  
            AND query_time >= DATE_SUB(NOW(), INTERVAL #{hours} HOUR)  
            AND temperature IS NOT NULL  
        GROUP BY DATE_FORMAT(query_time, '%Y-%m-%d %H:00:00')  
        ORDER BY hour DESC  
    </select>  
  
    <!-- 批量插入天气数据 -->  
    <insert id="batchInsert">  
        INSERT INTO weather_data  
        (location, location_name, temperature, humidity, wind_speed,  
        wind_direction, weather_desc, fx_time, query_time, raw_data)  
        VALUES  
        <foreach collection="list" item="item" separator=",">  
            (#{item.location}, #{item.locationName}, #{item.temperature},  
            #{item.humidity}, #{item.windSpeed}, #{item.windDirection},  
            #{item.weatherDesc}, #{item.fxTime}, #{item.queryTime},  
            #{item.rawData})  
        </foreach>  
    </insert>  
  
</mapper>
```

十、Service层实现

10.1 设计思路

Service层是业务逻辑的核心，遵循以下原则：

1. **单一职责**：每个Service专注于一个业务领域
2. **依赖注入**：通过构造器注入依赖（使用@RequiredArgsConstructor）
3. **事务管理**：写操作添加@Transactional注解
4. **异常处理**：捕获并记录异常，不向上抛出未处理异常
5. **日志记录**：关键操作记录INFO级别日志

10.2 MailService实现

java

```
/**
 * 邮件服务类
 *
 * 实现思路：
 * 1. 封装邮件发送逻辑
 * 2. 支持模板化邮件内容
 * 3. 提供异常处理，不影响主流程
 */
package com.example.service;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.MailException;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor // 为final字段生成构造器，实现依赖注入
@Slf4j // 提供log对象
public class MailService {

    // 使用final进行构造器注入（推荐方式）
    private final JavaMailSender mailSender;

    // 从配置文件注入邮件发送者邮箱
    @Value("${spring.mail.username}")
    private String fromEmail;

    // 从配置文件注入邮件主题
    @Value("${birthday.mail.subject}")
    private String subject;

    // 从配置文件注入邮件模板
    @Value("${birthday.mail.template}")
    private String template;

    /**
     * 发送生日祝福邮件
     *
     * 实现步骤：
     * 1. 构建邮件对象（SimpleMailMessage）
     * 2. 设置发件人、收件人、主题、正文
     * 3. 调用mailSender发送
     * 4. 记录成功/失败日志
     */
}
```



```

* 异常处理思路:
* - 捕获MailException, 记录错误日志
* - 不向外抛出异常, 避免影响其他用户
*
* @param to 收件人邮箱
* @param name 收件人姓名 (用于个性化称呼)
* @return 是否发送成功
*/
public boolean sendBirthdayEmail(String to, String name) {
    // 参数校验
    if (to == null || to.trim().isEmpty()) {
        log.error("邮件发送失败: 收件人邮箱为空");
        return false;
    }

    try {
        // 步骤1: 创建邮件消息对象
        SimpleMailMessage message = new SimpleMailMessage();

        // 步骤2: 设置发件人 (从配置文件读取)
        message.setFrom(fromEmail);

        // 步骤3: 设置收件人
        message.setTo(to);

        // 步骤4: 设置邮件主题
        message.setSubject(subject);

        // 步骤5: 设置邮件正文 (使用模板替换姓名)
        String content = String.format(template, name);
        message.setText(content);

        // 步骤6: 发送邮件
        mailSender.send(message);

        // 步骤7: 记录成功日志
        log.info("生日邮件发送成功 - 收件人: {}, 姓名: {}, 主题: {}",
            to, name, subject);
        return true;
    } catch (MailException e) {
        // 邮件发送失败处理
        log.error("生日邮件发送失败 - 收件人: {}, 姓名: {}, 错误类型: {}, 错误详情:
{}",
            to, name, e.getClass().getSimpleName(), e.getMessage(), e);
        return false;
    }
}

/**
* 发送自定义内容的邮件 (扩展功能)
*
* @param to 收件人
* @param subject 主题
* @param content 内容
*/
public boolean sendCustomEmail(String to, String subject, String content) {
    try {

```

```

        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom(fromEmail);
        message.setTo(to);
        message.setSubject(subject);
        message.setText(content);
        mailSender.send(message);
        log.info("自定义邮件发送成功 - 收件人: {}", to);
        return true;
    } catch (MailException e) {
        log.error("自定义邮件发送失败 - 收件人: {}", to, e);
        return false;
    }
}
}
}

```

10.3 WeatherService实现（核心）

java

```

/**
 * 天气服务类
 *
 * 实现思路：
 * 1. 天气数据获取：调用第三方API
 * 2. 数据解析：将JSON转为Java对象
 * 3. 数据存储：将解析后的数据存入数据库
 * 4. 数据查询：从数据库查询历史数据
 *
 * 这是整个项目的核心Service，实现了天气数据的完整生命周期管理
 */
package com.example.service;

import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.example.dto.WeatherInfo;
import com.example.dto.WeatherResponse;
import com.example.entity.WeatherData;
import com.example.mapper.WeatherMapper;
import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.http.client.config.RequestConfig;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;

```

```

@Service
@RequiredArgsConstructor
@Slf4j
public class WeathersService {

    // ===== 依赖注入 =====
    private final WeatherMapper weatherMapper;

    // ===== 配置注入 =====
    @Value("${weather.api.url}")
    private String apiUrl;

    @Value("${weather.api.key}")
    private String apiKey;

    @Value("${weather.api.location}")
    private String location;

    @Value("${weather.api.location-name}")
    private String locationName;

    @Value("${weather.api.connect-timeout:10000}")
    private int connectTimeout;

    @Value("${weather.api.read-timeout:10000}")
    private int readTimeout;

    // ===== 工具类 =====
    private final ObjectMapper objectMapper = new ObjectMapper();
    private static final DateTimeFormatter FORMATTER =
    DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    private static final DateTimeFormatter API_DATE_FORMATTER =
    DateTimeFormatter.ISO_OFFSET_DATE_TIME;

    // ===== 核心业务方法 =====

    /**
     * 获取天气数据并存储到数据库
     *
     * 实现思路:
     * 1. 构建API请求URL
     * 2. 发送HTTP请求获取数据
     * 3. 解析JSON数据
     * 4. 构建WeatherData实体
     * 5. 插入数据库
     * 6. 返回存储后的实体（包含自增ID）
     *
     * 事务说明: @Transactional确保数据一致性,
     * 如果插入失败会回滚, 但API调用不在事务内
     *
     * @return 存储后的天气数据实体
     */
    @Transactional(rollbackFor = Exception.class)
    public WeatherData fetchAndSaveWeather() {
        log.info("开始获取天气数据...");
        long startTime = System.currentTimeMillis();

        try {

```

```

// ----- 步骤1: 构建API请求URL -----
String url = buildApiUrl();
log.debug("请求URL: {}", url);

// ----- 步骤2: 调用天气API -----
String responseBody = callWeatherApi(url);
log.debug("API响应: {}", responseBody.length() > 200 ?
    responseBody.substring(0, 200) + "...": responseBody);

// ----- 步骤3: 解析并构建实体对象 -----
WeatherData weatherData = parseAndBuildWeatherData(responseBody);

// ----- 步骤4: 插入数据库 -----
int insertCount = weatherMapper.insert(weatherData);

long duration = System.currentTimeMillis() - startTime;

if (insertCount > 0) {
    log.info("天气数据存储成功 - ID: {}, 温度: {}°C, 湿度: {}%, 耗时:
{}ms",

        weatherData.getId(),
        weatherData.getTemperature(),
        weatherData.getHumidity(),
        duration);
    return weatherData;
} else {
    log.warn("天气数据插入失败, 影响行数为0");
    throw new RuntimeException("数据插入失败");
}

} catch (Exception e) {
    log.error("获取天气数据失败: {}", e.getMessage(), e);

    // ----- 失败时记录错误信息到数据库 -----
    WeatherData errorData = createErrorRecord(e.getMessage());
    weatherMapper.insert(errorData);

    return errorData;
}

}

/**
 * 获取实时天气信息（不存储，仅返回）
 *
 * 实现思路：
 * 1. 直接调用API
 * 2. 解析并返回WeatherInfo对象
 * 3. 用于Controller查询当前天气
 *
 * @return 天气信息DTO
 */
public WeatherInfo getWeatherInfo() {
    try {
        String url = buildApiUrl();
        String responseBody = callWeatherApi(url);
        return parseWeatherObject(responseBody);
    } catch (Exception e) {
        log.error("获取天气信息失败", e);
    }
}

```

```

        return weatherInfo.builder()
            .success(false)
            .errorMsg(e.getMessage())
            .queryTime(LocalDateTime.now())
            .build();
    }
}

// ===== 数据查询方法 =====

/**
 * 查询最新的天气数据（从数据库）
 */
public WeatherData getLatestWeather() {
    return weatherMapper.findLatestByLocation(location);
}

/**
 * 查询最近24小时的天气数据
 */
public List<WeatherData> getLast24HourWeather() {
    LocalDateTime endTime = LocalDateTime.now();
    LocalDateTime startTime = endTime.minusHours(24);
    return weatherMapper.findByTimeRange(location, startTime, endTime);
}

/**
 * 查询最近N条天气记录
 */
public List<WeatherData> getRecentWeather(int limit) {
    return weatherMapper.findRecentRecords(location, limit);
}

/**
 * 查询指定时间范围内的天气数据
 */
public List<WeatherData> getWeatherByTimeRange(LocalDateTime start,
LocalDateTime end) {
    return weatherMapper.findByTimeRange(location, start, end);
}

/**
 * 删除指定时间之前的旧数据
 *
 * @param beforeTime 时间阈值
 * @return 删除的记录数
 */
@Transactional(rollbackFor = Exception.class)
public int deleteOldRecords(LocalDateTime beforeTime) {
    LambdaQueryWrapper<WeatherData> wrapper = new LambdaQueryWrapper<>();
    wrapper.lt(WeatherData::getQueryTime, beforeTime);

    int deletedCount = weatherMapper.delete(wrapper);
    log.info("清理天气数据 - 删除 {} 条 {} 之前的记录",
        deletedCount, beforeTime.format(FORMATTER));

    return deletedCount;
}

```

```
// ===== 私有辅助方法 =====

/**
 * 构建API请求URL
 */
private String buildApiUrl() {
    return String.format("%s?location=%s&key=%s", apiUrl, location, apiKey);
}

/**
 * 调用天气API
 *
 * 实现细节:
 * 1. 配置超时时间（连接超时、读取超时）
 * 2. 设置请求头Accept: application/json
 * 3. 检查HTTP状态码
 * 4. 使用try-with-resources自动关闭资源
 *
 * @param url 请求URL
 * @return 响应字符串
 */
private String callWeatherApi(String url) throws Exception {
    // 配置请求参数
    RequestConfig requestConfig = RequestConfig.custom()
        .setConnectTimeout(connectTimeout) // 连接超时
        .setSocketTimeout(readTimeout) // 读取超时
        .setConnectionRequestTimeout(connectTimeout)
        .build();

    // 创建HttpClient（使用try-with-resources自动关闭）
    try (CloseableHttpClient httpClient = HttpClients.custom()
        .setDefaultRequestConfig(requestConfig)
        .build()) {

        HttpGet httpGet = new HttpGet(url);
        httpGet.setHeader("Accept", "application/json");
        httpGet.setHeader("User-Agent", "SpringBoot-ScheduledTask/1.0");

        log.debug("发送HTTP请求: {}", url);

        // 执行请求
        try (CloseableHttpResponse response = httpClient.execute(httpGet)) {
            int statusCode = response.getStatusLine().getStatusCode();
            String responseBody =
                EntityUtils.toString(response.getEntity());

            log.debug("HTTP响应状态码: {}", statusCode);

            // 检查HTTP状态码
            if (statusCode != 200) {
                throw new RuntimeException(String.format(
                    "API返回错误状态码: %d, 响应: %s", statusCode,
                    responseBody));
            }

            return responseBody;
        }
    }
}
```

```

    }
}

/**
 * 解析并构建WeatherData实体
 *
 * 解析流程:
 * 1. JSON反序列化为WeatherResponse对象
 * 2. 从hourly列表中取第一条作为当前天气
 * 3. 提取温度、湿度、风速、风向等字段
 * 4. 设置查询时间等元数据
 *
 * @param rawData API返回的原始JSON
 * @return WeatherData实体
 */
private WeatherData parseAndBuildWeatherData(String rawData) throws
Exception {
    WeatherResponse response = objectMapper.readValue(rawData,
WeatherResponse.class);

    WeatherData weatherData = new WeatherData();

    // 基础信息
    weatherData.setLocation(location);
    weatherData.setLocationName(locationName);
    weatherData.setRawData(rawData);
    weatherData.setQueryTime(LocalDateTime.now());

    // 检查响应是否成功
    if (!"200".equals(response.getCode())) {
        log.warn("API返回非成功状态码: {}", response.getCode());
        return weatherData;
    }

    // 解析天气数据
    if (response.getHourly() != null && !response.getHourly().isEmpty()) {
        WeatherResponse.HourlyWeather current = response.getHourly().get(0);

        // 温度解析 (字符串转BigDecimal)
        if (current.getTemp() != null && !current.getTemp().isEmpty()) {
            try {
                weatherData.setTemperature(new
BigDecimal(current.getTemp()));
            } catch (NumberFormatException e) {
                log.warn("温度解析失败: {}", current.getTemp

```